

Universidade do Vale do Itajaí
Disciplina: Sistemas Operacionais
Professor: Michael Douglas C A
Trabalho M2 – Escalonadores

1. Esta avaliação deve ser feita, em dupla ou trio. Trabalhos individuais não serão aceitos.
 2. Data de entrega: 06/06/2026 até 23:59. Não aceitos trabalhos entregues em atraso.
 3. Esta avaliação tem por objetivo consolidar o aprendizado sobre escalonadores.
 4. A implementação deverá ser desenvolvida utilizando python, c++ ou Java. O uso de bibliotecas de terceiros deverão ser explicado com uma justificativa válida que ficará ao critério do professor aceitar ou não.
 5. O sistema deve ser entregue funcionando corretamente.
 6. Deve ser apresentado um relatório eletrônico em formato PDF (em outro formato é descontado 2,0 pontos) que contenha: a. Identificação dos autores e do trabalho. b. Enunciado do projeto. c. Explicação e contexto da aplicação para compreensão do problema tratado pela solução. d. Resultados/simulações. e. Trechos de códigos pertinentes da solução. f. Análise e discussão sobre os resultados finais.
 7. Seguindo a mesma diretriz do trabalho anterior, deverá ser disponibilizado os códigos da implementação juntamente com o relatório (salvo o caso da disponibilidade em repositório aberto do aluno, que deve ser fornecido o link). Também deve ser apresentado o trabalho em aula para o professor. Na apresentação do trabalho, não é necessário utilizar slides, apenas apresentar o código e sua execução (compilar na apresentação)
- Obs: poderá ser solicitado que apenas um dos integrantes apresente a solução!

Desenvolver um simulador de tempo discreto (orientado a tiques de relógio) que integre um escalonador de CPU Round Robin e um gerenciador de memória virtual com paginação sob demanda, utilizando o algoritmo de substituição de páginas LRU (Least Recently Used).

Regras do Simulador

- **O Relógio (Clock):** O sistema começa no tempo 0. A cada iteração do laço principal, o tempo avança 1 unidade (1 tique).
- **Escalonamento (Round Robin):**
 - O processo no topo da Fila de Prontos ganha a CPU.
 - A cada tique do relógio, ele acessa a próxima página de sua lista de requisições.
 - Se o processo usar a CPU pelo tempo igual ao **Quantum**, ele sofre preempção e vai para o final da Fila de Prontos.
- **Gerenciamento de Memória:**
 - Antes de consumir 1 tique de CPU, o sistema verifica se a página solicitada pelo processo está na RAM.
 - **RAM Hit:** Se a página estiver na RAM, o processo executa normalmente por 1 tick. O LRU é atualizado (esta página passa a ser a mais recentemente usada).
 - **Page Fault (RAM Miss):** Se a página não estiver na RAM, o processo sofre interrupção imediata, independentemente do seu Quantum.
- **Tratamento do Page Fault:**
 - O processo é movido para a **Fila de Bloqueados** e deve permanecer lá pelo tempo exato da **Penalidade de I/O** definida no arquivo.
 - O sistema deve carregar a página na RAM. Se a RAM estiver cheia, o algoritmo **LRU** deve escolher a página de qualquer processo que não é acessada há mais tempo e removê-la para dar espaço.
 - Após cumprir o tempo de penalidade, o processo volta para o final da Fila de Prontos para tentar acessar aquela página novamente.

Saída Esperada do Programa

O simulador não precisa de interface gráfica. Ele deve imprimir no console um relatório final com:

1. **Tempo de Retorno :** O tempo total desde a chegada do processo até ele concluir o acesso à sua última página.
2. **Total de Page Faults:** Quantas falhas de página cada processo sofreu.
3. **Log de Execução:** Imprimir os eventos principais (ex: [Tempo 4] P1 sofreu Page Fault na página 5).

A Integração: RR com LRU

Fluxo do cenário desejado:

- O **Quantum do RR** é 2 (o processo pode usar a CPU por 2 turnos seguidos).
- A **Memória RAM** só tem **2 espaços** no total.

O P1 entra na CPU (graças ao escalonador Round Robin) e o tempo começa a rodar.

Turno 1 na CPU

- **Ação do P1:** "Eu quero ler o meu primeiro número do **.txt**: a **página 1**."
- **O Encontro:** O sistema olha para a Memória RAM. A página 1 está lá? **Não.** * **O que acontece? (Page Fault):** 1. O escalonador **RR** diz: "P1, você não pode continuar. Você perde a CPU agora mesmo e vai para a fila de Bloqueados, esperando o disco rígido buscar essa página." 2. O **LRU** diz: "Ok, vou buscar a página 1 e colocar na RAM."
- **Estado da RAM:** [1] [Vazio]

(P1 fica um tempo de bloqueados. Quando o bloqueados acaba, ele volta para a fila de Prontos e, eventualmente, ganha a CPU de novo).

Turno 2 na CPU (P1 tenta de novo)

- **Ação do P1:** "Eu quero ler o meu primeiro número: a **página 1**."
- **O Encontro:** A página 1 está na RAM? **Sim!**
- **O que acontece? (Hit):**
 1. O **LRU** diz: "Anotado. A página 1 acabou de ser usada, então ela é a mais 'fresquinha' da memória."
 2. O **RR** diz: "Beleza, P1. Você conseguiu executar por 1 tique de tempo. Como o Quantum é 2, você ainda tem mais 1 tique de tempo sobrando. Pode tentar ler seu próximo número."

Turno 3 na CPU (Continuação do Quantum)

- **Ação do P1:** "Agora quero ler meu segundo número do **.txt**: a **página 2**."
- **O Encontro:** A página 2 está na RAM? **Não.**
- **O que acontece? (Page Fault):**
 1. O **RR** diz: "Pausa! Para a fila de bloqueado de novo."
 2. O **LRU** busca a página 2 e coloca no espaço vazio.
- **Estado da RAM:** [1] [2]

(P1 volta do bloqueado e consegue executar a página 2 com sucesso. O LRU anota que a página 2 é a mais 'frequente' e a página 1 ficou mais velha).

Turno 4 na CPU (O momento em que o LRU brilha)

- **Ação do P1:** "Agora quero ler meu último número: a **página 5**."

- **O Encontro:** A página 5 está na RAM? **Não.** (A RAM tem a 1 e a 2).
- **O que acontece? (Page Fault com RAM Cheia):**
 1. O **RR** manda o P1 para o bloqueado de novo.
 2. O **LRU** olha para a RAM e diz: "Estou sem espaço! Preciso jogar alguém fora. Quem eu jogo?"
 3. Ele analisa: "A página 2 foi usada agorinha. A página 1 não é usada há mais tempo."
 4. **Ação do LRU:** Joga a página 1 no lixo e coloca a página 5 no lugar.
- **Estado da RAM:** [5] [2]